Technical Reflection Report

DevOps, automated testing and containerisation in the TechMove system

Mahlatsi Ramano · ST10209387 Repository: github.com/MahlatsiRAMANO/TechMove

1. Introduction

In Part 3 the TechMove prototype was split from a single ASP.NET Core MVC application into three cooperating services: an ASP.NET Core Web API that owns the business logic and the SQL Server database, an MVC frontend that only talks to the API over HTTP, and the database itself. Each runs in its own Docker container and the whole system is started with one docker-compose.yml file. This reflection explains why automated testing is essential once a system is delivered through a CI/CD pipeline, and how Docker solves the classic "it works on my machine" problem, using concrete examples from building TechMove.

2. DevOps and testing: why automated tests are critical in CI/CD

2.1 The problem automation solves

Splitting the monolith multiplied the number of things that can break. In Part 2 a controller called a service in the same process; now the MVC app sends a JSON request across a network to an API, which validates a JWT, applies the workflow and writes to a database in another container. A single renamed property, a changed status code or an enum serialised as a number instead of a string would compile perfectly in both projects and still break the system at runtime. Manual testing cannot keep up with this: clicking through every screen after every commit is slow, inconsistent and easy to skip under deadline pressure. Continuous Delivery depends on the idea that every change is proven releasable by an automated process rather than by someone's memory (Humble and Farley, 2010).

2.2 How the TechMove pipeline stops bugs reaching production

The GitHub Actions workflow runs on every push and acts as a series of quality gates. A commit can only be considered deployable if it passes every stage:

Stage	What it proves
1. Build	Both projects and the test project compile on a clean Linux machine (not just my Windows laptop).
2. Unit tests	Business rules in isolation: USD→ZAR maths and rounding, PDF validation (.exe, renamed .exe, empty, oversize), the contract state machine, the LINQ filter, and the services with Moq fakes.
3. In-process integration tests	The real API pipeline – routing, JWT authentication, model validation, controllers, services, repositories and JSON – using WebApplicationFactory with an in-memory database.
4. Docker integration tests	docker compose up --build starts SQL Server, the API and the frontend; the same integration tests then call the running API container on localhost:5001.

Because the stages run in this order, failures are caught where they are cheapest to fix. This follows the test pyramid: many fast unit tests at the base, fewer but broader integration tests above them (Fowler, 2012). If any stage fails, the commit is marked red on GitHub and nothing from it should be released.

2.3 Integration tests check behaviour, not just “not null”

The integration tests were written to verify the contract between the frontend and the API, because that is exactly what the decoupling put at risk. Each test asserts the HTTP status code and the data. For example, the contract test posts a multipart form with a real PDF, expects 201 Created with a Location header, reads the contract back through that Location, and checks every field and the downloaded PDF byte for byte. That one test proves routing, file validation, the file server, the database write, the read path and the DTO mapping together. Other tests check the failure paths that users actually hit: 400 when a service request is made against an Expired or On Hold contract, 403 when a Staff user tries to approve a contract, 401 with a missing or tampered token, and 503 – with nothing saved – when the external currency API is down. Microsoft recommends this style of test for ASP.NET Core because it exercises the app’s real middleware and dependency-injection configuration (Microsoft, 2025).

2.4 Concrete bugs this approach prevents

- **Breaking API changes:** if someone renames ContractDto.Status or returns 200 instead of 201, the create-then-read tests fail before the MVC app (and the user) ever see it.
- **Security regressions:** removing an [Authorize] attribute would make the 401 and 403 tests fail immediately.
- **Business-rule regressions:** if a refactor let service requests through on Expired contracts, the workflow unit tests and the 400 integration tests both fail.
- **Environment problems:** a missing migration, a wrong connection string or a container that cannot reach SQL Server fails the Docker job, which a developer running only in Visual Studio would never notice.

Research on high-performing teams links exactly these practices – automated testing and continuous integration – to both faster delivery and lower change-failure rates (Forsgren, Humble and Kim, 2018). For TechMove the practical benefit is confidence: I can refactor the service layer, push, and know within minutes whether anything a user depends on has changed.

3. Containerisation: solving “it works on my machine”

3.1 The problem as I experienced it

This problem was not theoretical in this project. During Part 2 the application only worked on my laptop after several environment-specific fixes: the project targeted .NET 8 while my machine only had the .NET 10 runtime, so the EF Core tools refused to run; SQL Server LocalDB would not start on my Windows 11 machine; and the Azure SQL database rejected connections every time my IP address changed. None of these were bugs in the code – they were differences between environments. A lecturer or a production server would have hit a different set of problems.

3.2 How Docker guarantees consistency

A Docker image packages the application together with everything it needs to run – the exact .NET runtime, the published application files and the operating-system libraries – into one immutable artefact (Docker, n.d.). TechMove uses multi-stage Dockerfiles: the first stage uses the full .NET 10 SDK image to restore and publish the project, and the final stage copies only the published output into the much smaller ASP.NET runtime image, running as a non-root user. The project file is copied and restored before the source code, so the NuGet layer is cached and rebuilt only when dependencies change.

docker-compose.yml then describes the whole environment as code: three containers (sql-server-db, glms-backend-api and glms-frontend-web), a private bridge network, named volumes for the database and uploaded agreements, and all configuration passed in as environment variables. The containers find each other by service name through Docker's internal DNS, so the frontend's Api__BaseUrl is simply http://glms-backend-api:8080/ and the API's connection string uses Server=sql-server-db. The API waits for the database health check and applies its EF Core migration on start-up, so a fresh machine needs only one command: docker compose up --build.

Concern	Before (Part 2)	With Docker (Part 3)
.NET version	Whatever was installed locally	Pinned in the image (sdk:10.0 / aspnet:10.0)
Database	LocalDB or Azure, different per machine	SQL Server 2022 container, identical everywhere
Configuration	Edited by hand in appsettings.json	Environment variables in docker-compose.yml
Networking	localhost ports, firewalls	Private network, services addressed by name
Setting up a new machine	Hours of troubleshooting	docker compose up --build

3.3 Dev, Test and Prod use the same artefact

The most important consequence is that the image tested is the image deployed. The GitHub Actions Docker job builds the same Dockerfiles, starts the same compose file and runs the integration tests against those containers. Only configuration values such as the SA password and JWT signing key change between environments, and they are supplied from outside the image (an .env file locally, secrets in a real deployment) rather than committed to source control – which also removed the Azure credentials that had previously been stored in appsettings.json.

4. Linking testing and containers

Automated testing and containerisation solve two halves of the same problem. Tests answer "does the code behave correctly?", while containers answer "will it behave the same way somewhere else?". Running the integration suite against the Docker containers in CI combines the two: every push proves that the exact images, wired together exactly as they will be in production, still create, read and protect TechMove's data correctly. That is what allows a team to release small changes frequently without fear, which is the core promise of DevOps (Kim et al., 2016).

5. Conclusion

Decoupling TechMove into an API and a frontend made the system easier to scale and change independently, but it also introduced network boundaries where silent failures can occur. Automated unit and integration tests in a CI pipeline turn those boundaries into checked contracts, and Docker removes the environment differences that caused most of my practical problems in Part 2. Together they mean that a working build on GitHub is strong evidence of a working system anywhere it runs.

References

Docker (n.d.) *What is a container?* Available at: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/ (Accessed: 29 October 2026).

Forsgren, N., Humble, J. and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps*. Portland, OR: IT Revolution Press.

Fowler, M. (2012) *TestPyramid*. Available at: https://martinfowler.com/bliki/TestPyramid.html (Accessed: 29 October 2026).

Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.

Kim, G., Humble, J., Debois, P. and Willis, J. (2016) *The DevOps Handbook*. Portland, OR: IT Revolution Press.

Microsoft (2025) *Integration tests in ASP.NET Core*. Available at: https://learn.microsoft.com/aspnet/core/test/integration-tests (Accessed: 29 October 2026).